

LAPORAN TUGAS BESAR 2 PEMBELAJARAN MESIN

IF3270 Pembelajaran Mesin

Convolutional Neural Network dan Recurrent Neural Network



Disusun Oleh:

Indah Novita Tangdililing (13523047)

Muhammad Fithra Rizki (13523049)

Muhammad Timur Kanigara (13523055)

SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA

INSTITUT TEKNOLOGI BANDUNG

BANDUNG

2026

DAFTAR ISI

DAFTAR ISI.....	1
BAB I	
DESKRIPSI PERSOALAN.....	2
BAB II	
PEMBAHASAN.....	3
2.1 Penjelasan Implementasi.....	3
2.1.1 Implementasi CNN.....	3
2.1.2 Implementasi RNN/LSTM untuk Image Captioning.....	4
2.1.3 Utilitas untuk Preprocessing dan Evaluasi.....	6
2.2 Penjelasan Forward Propagation.....	7
2.2.1 Forward Propagation CNN.....	7
2.2.2 Forward Propagation RNN.....	9
2.2.3 Forward Propagation LSTM.....	11
2.3 Implementasi Bonus.....	13
2.3.1 Visualisasi Fitur CNN.....	13
2.3.2 Beam Search Decoder.....	13
2.3.3 Image Captioning dengan Init-Inject.....	14
2.3.4 Batch Inference.....	15
2.3.5 Backward Propagation from Scratch.....	15
2.4 Hasil Pengujian.....	16
2.4.1 CNN.....	16
2.4.2 Simple RNN dan LSTM untuk Image Captioning.....	23
BAB III	
KESIMPULAN DAN SARAN.....	35
3.1 Kesimpulan.....	35
3.2 Saran.....	36
Pembagian Tugas Tiap Anggota Kelompok.....	38
Referensi.....	39

BAB I

DESKRIPSI PERSOALAN

Tugas Besar 2 IF3270 Pembelajaran Mesin membahas implementasi dan analisis model deep learning berbasis Convolutional Neural Network (CNN), Recurrent Neural Network (RNN), dan Long Short-Term Memory (LSTM). Fokus utama tugas ini adalah memahami bagaimana proses forward propagation bekerja pada arsitektur deep learning melalui implementasi menggunakan Keras serta implementasi from scratch berbasis NumPy.

Pada bagian pertama, permasalahan yang diselesaikan adalah klasifikasi citra menggunakan dataset Intel Image Classification yang terdiri atas enam kategori gambar, yaitu buildings, forest, glacier, mountain, sea, dan street. Model CNN digunakan untuk mempelajari pola visual pada gambar sehingga mampu melakukan klasifikasi dengan baik. Selain melakukan pelatihan model menggunakan Keras, dilakukan juga implementasi layer-layer CNN seperti Conv2D, pooling, flatten, dan fungsi aktivasi secara from scratch untuk memahami mekanisme kerja jaringan saraf konvolusional.

Pada bagian kedua, permasalahan yang dibahas adalah image captioning menggunakan dataset Flickr8k. Pada tugas ini, CNN digunakan sebagai encoder untuk mengekstraksi fitur gambar, sedangkan RNN dan LSTM digunakan sebagai decoder untuk menghasilkan caption secara berurutan berdasarkan isi gambar. Implementasi dilakukan menggunakan arsitektur encoder-decoder dengan pendekatan pre-inject, di mana feature vector dari CNN dimasukkan ke jaringan sekuensial sebelum token caption diproses.

Selain implementasi model menggunakan Keras, tugas ini juga menuntut implementasi forward propagation from scratch untuk Embedding layer, SimpleRNN, dan LSTM menggunakan NumPy. Berbagai eksperimen dilakukan untuk membandingkan performa model berdasarkan variasi hyperparameter, arsitektur, serta implementasi Keras dan from scratch. Evaluasi dilakukan menggunakan metrik seperti macro F1-score, BLEU-4, dan METEOR score untuk mengetahui kualitas model pada tugas klasifikasi citra maupun image captioning.

BAB II

PEMBAHASAN

2.1 Penjelasan Implementasi

Implementasi tugas besar ini dibagi menjadi dua bagian utama, yaitu modul CNN untuk klasifikasi gambar dan modul RNN/LSTM untuk image captioning. Secara umum, implementasi menggunakan dua pendekatan: model berbasis Keras untuk proses training utama, serta implementasi from scratch berbasis NumPy untuk menunjukkan pemahaman terhadap proses forward propagation dan sebagian backward propagation.

2.1.1 Implementasi CNN

Modul CNN digunakan untuk melakukan klasifikasi gambar Intel Image Classification ke dalam enam kelas, yaitu buildings, forest, glacier, mountain, sea, dan street.

Kelas `CNNExperimentConfig` digunakan untuk menyimpan konfigurasi eksperimen CNN. Atribut pada kelas ini meliputi `experiment_id`, `conv_depth`, `filters_base`, `kernel_size`, `pooling`, `image_size`, `num_classes`, `dense_units`, dan `learning_rate`. Properti `filters` menghasilkan daftar jumlah filter untuk setiap layer konvolusi berdasarkan kedalaman model. Method `to_dict()` digunakan untuk mengubah konfigurasi menjadi dictionary agar dapat disimpan ke file JSON.

Kelas `SimpleLocallyConnected2D` merupakan implementasi fallback untuk layer `Locally Connected 2D`. Berbeda dengan `Conv2D` biasa, layer ini tidak menggunakan weight sharing, sehingga setiap posisi spasial memiliki kernel sendiri. Atribut pentingnya adalah `filters`, `kernel_size`, `strides`, `activation`, `kernel`, dan `bias`. Method `build()` membuat bobot berdasarkan ukuran input, sedangkan `call()` melakukan ekstraksi patch menggunakan `tf.image.extract_patches`, menghitung output dengan `einsum`, menambahkan bias, lalu menerapkan fungsi aktivasi. Method `get_config()` digunakan agar layer dapat disimpan dan dimuat kembali oleh Keras.

Fungsi `build_shared_cnn()` membangun model CNN dengan layer `Conv2D`, `pooling`, `flatten`, `dense hidden layer`, dan `output softmax`. Model ini menggunakan weight sharing pada kernel konvolusi. Fungsi `build_non_shared_cnn()` membangun arsitektur serupa, tetapi mengganti `Conv2D` dengan `LocallyConnected2D` atau `SimpleLocallyConnected2D`, sehingga bobot tidak dibagi antar posisi gambar.

Pada implementasi from scratch, kelas `Conv2D` merepresentasikan layer konvolusi berbasis NumPy. Atributnya meliputi `kernel`, `bias`, `strides`, `padding`, dan `activation`. Method

forward() menerima input gambar, melakukan padding bila diperlukan, menggeser kernel pada setiap posisi, menghitung hasil konvolusi, menambahkan bias, dan menerapkan aktivasi. Method backward() menghitung gradien terhadap input, kernel, dan bias.

Kelas LocallyConnected2D mengimplementasikan layer konvolusi tanpa weight sharing. Atribut utamanya adalah kernel, bias, kernel_size, strides, padding, dan activation. Method forward() mengambil patch pada setiap posisi, lalu mengalikan patch tersebut dengan kernel khusus untuk posisi itu. Method backward() menghitung gradien input, kernel lokal, dan bias.

Kelas Pooling2D menjadi kelas dasar untuk operasi pooling. Atributnya terdiri dari pool_size, strides, padding, dan mode. Method forward() melakukan pooling pada setiap region input, baik dengan mode maksimum maupun rata-rata. Method backward() menyebarkan gradien kembali ke input. Kelas MaxPooling2D adalah turunan dengan mode max, sedangkan AveragePooling2D menggunakan mode avg.

Kelas GlobalAveragePooling2D dan GlobalMaxPooling2D digunakan untuk mereduksi feature map menjadi vektor dengan mengambil rata-rata atau maksimum pada seluruh dimensi spasial. Keduanya memiliki method forward() untuk menghitung output dan backward() untuk menghitung gradien input.

Kelas Flatten mengubah input berdimensi tiga atau empat menjadi vektor satu dimensi atau batch vektor. Method forward() menyimpan bentuk input asli, sedangkan backward() mengembalikan gradien ke bentuk semula.

Kelas Dense pada modul CNN from scratch merepresentasikan fully connected layer. Atributnya adalah kernel, bias, dan activation. Method forward() menghitung operasi linear $x @ \text{kernel} + \text{bias}$, kemudian menerapkan aktivasi. Method backward() menghitung gradien terhadap input, kernel, dan bias.

Kelas SequentialScratchModel digunakan sebagai pembungkus model berurutan. Atribut utamanya adalah layers, yaitu daftar layer yang akan dieksekusi. Method forward() menjalankan setiap layer secara berurutan, predict() menjadi alias dari forward(), dan backward() melakukan propagasi balik dari layer terakhir ke layer pertama.

2.1.2 Implementasi RNN/LSTM untuk Image Captioning

Modul RNN/LSTM digunakan untuk menghasilkan caption berdasarkan fitur gambar Flickr8k. Gambar terlebih dahulu diekstraksi menjadi feature vector menggunakan encoder CNN pretrained seperti InceptionV3 atau VGG16. Feature vector tersebut kemudian diberikan ke decoder berbasis RNN atau LSTM.

Kelas `EmbeddingLayer` digunakan untuk mengubah indeks kata menjadi vektor embedding. Atributnya adalah `W`, yaitu matriks embedding berukuran (`vocab_size`, `embed_dim`), dan `_last_input` untuk menyimpan input terakhir saat backward. Method `load_weights()` mengambil bobot dari layer Keras, `set_weights()` mengisi bobot secara manual, `forward()` mengambil embedding berdasarkan indeks token, dan `backward()` menghitung gradien terhadap matriks embedding.

Kelas `DenseLayer` pada modul RNN/LSTM merepresentasikan fully connected layer sederhana. Atributnya adalah `W`, `b`, dan `_act` sebagai fungsi aktivasi. Method `load_weights()` dan `set_weights()` digunakan untuk mengisi bobot, `forward()` menghitung output linear dan aktivasi, sedangkan `backward()` menghitung gradien untuk aktivasi, bobot, bias, dan input. Aktivasi yang didukung meliputi `relu`, `softmax`, `sigmoid`, `tanh`, dan `linear`.

Kelas `SimpleRNNCell` merepresentasikan satu cell RNN. Atributnya adalah `Wx`, `Wh`, dan `b`. Method `forward()` menghitung hidden state baru menggunakan persamaan $\tanh(xW_x + h_{prev}W_h + b)$. Method `forward_with_cache()` menyimpan nilai antara untuk kebutuhan backward, sedangkan `backward()` menghitung gradien terhadap input, hidden state sebelumnya, dan parameter cell. Properti `hidden_size` mengembalikan ukuran hidden state.

Kelas `SimpleRNNDecoder` menyusun satu atau lebih `SimpleRNNCell` menjadi decoder berlapis. Atributnya adalah `cells` dan `n_layers`. Method `forward()` memproses seluruh sequence token, `forward_with_cache()` menjalankan forward sambil menyimpan cache, `backward()` melakukan backpropagation through time, dan `step()` memproses satu token pada satu timestep. Method `step()` digunakan saat proses generasi caption secara autoregresif.

Kelas `LSTMCell` mengimplementasikan satu cell LSTM. Atributnya adalah `Wx`, `Wh`, `b`, dan `_hidden_size`. Method `forward()` menghitung empat gate LSTM, yaitu input gate, forget gate, candidate gate, dan output gate. Cell state diperbarui dengan $c_t = f * c_{prev} + i * g$, lalu hidden state dihitung dengan $h_t = o * \tanh(c_t)$. Method `forward_with_cache()` menyimpan nilai gate untuk backward, sedangkan `backward()` menghitung gradien terhadap input, hidden state, cell state, dan seluruh parameter.

Kelas `LSTMDecoder` menyusun beberapa `LSTMCell` menjadi decoder sequence. Atributnya adalah `cells` dan `n_layers`. Method `forward()` memproses sequence lengkap dan menghasilkan output setiap timestep. Method `forward_with_cache()` menyimpan cache untuk pelatihan from scratch. Method `backward()` melakukan backpropagation through time dengan memperhitungkan hidden state dan cell state. Method `step()` digunakan untuk inferensi satu token pada satu waktu.

Kelas `CaptionModel` merupakan model utama untuk caption generation from scratch. Atributnya meliputi `decoder_type`, `injection_method`, `embedding`, `image_projection`, `decoder`, `output_layer`, `word2idx`, `idx2word`, dan `n_layers`. `decoder_type` menentukan apakah model menggunakan RNN atau LSTM. `injection_method` menentukan cara memasukkan fitur gambar, yaitu `pre` atau `init`. Pada metode `pre`, fitur gambar diproyeksikan ke ruang embedding dan diberikan sebagai token awal sebelum caption. Pada metode `init`, fitur gambar digabungkan dengan output decoder pada setiap timestep.

Method `load_from_keras()` pada `CaptionModel` memuat bobot dari model Keras ke implementasi NumPy. Method `load_weights_manual()` digunakan untuk pengujian dengan bobot manual. Method `generate_caption()` menghasilkan caption menggunakan greedy decoding, yaitu selalu memilih token dengan probabilitas terbesar. Method `generate_caption_beam()` menggunakan beam search untuk mempertahankan beberapa kandidat caption terbaik. Method `forward_sequence()` menjalankan forward untuk satu sequence, sedangkan `forward_sequence_batch()` menjalankan forward untuk batch. Method `generate_captions_batch()` menghasilkan caption untuk banyak gambar secara bertahap.

Kelas `DecoderExperimentConfig` menyimpan konfigurasi eksperimen decoder. Atributnya meliputi `decoder_type`, `n_layers`, `hidden_size`, `embed_dim`, `max_seq_len`, `feature_dim`, `learning_rate`, dan `injection_method`. Properti `experiment_id` membentuk nama eksperimen berdasarkan konfigurasi, sedangkan `to_dict()` mengubah konfigurasi menjadi dictionary.

Fungsi `build_decoder_model()` membangun model decoder berbasis Keras. Input pertama adalah fitur gambar, sedangkan input kedua adalah token caption. Fitur gambar diproyeksikan menggunakan dense layer `image_projection`, sementara token caption masuk ke layer `Embedding`. Setelah itu, sequence diproses oleh satu atau lebih layer `SimpleRNN` atau `LSTM`. Output akhir dipetakan ke vocabulary menggunakan dense softmax bernama `output_dense`.

Fungsi `masked_loss()` menghitung sparse categorical cross entropy dengan mengabaikan token padding. Fungsi `masked_accuracy()` menghitung akurasi hanya pada token non-padding. Fungsi `build_dataset()` membentuk dataset TensorFlow dari pasangan fitur gambar, input token, dan target token.

2.1.3 Utilitas untuk Preprocessing dan Evaluasi

Modul `text_utils` digunakan untuk pemrosesan caption. Fungsi `clean_caption()` membersihkan teks dengan lowercasing dan menghapus karakter non-huruf. Fungsi `build_vocab()` membangun vocabulary dari caption. Fungsi `encode_caption()` mengubah caption

menjadi indeks token dengan token khusus <start> dan <end>. Fungsi `decode_caption()` mengubah indeks kembali menjadi teks. Fungsi `prepare_training_pair()` membentuk pasangan input-target untuk training decoder.

Modul `feature_extraction` digunakan untuk mengekstraksi fitur gambar. Fungsi `build_inception_encoder()` dan `build_vgg16_encoder()` membuat encoder pretrained tanpa classification head. Fungsi `extract_features()` memproses gambar secara batch dan menghasilkan dictionary fitur berdasarkan nama file. Fungsi `save_features()` dan `load_features()` menyimpan serta memuat fitur dalam format `.npy`.

Evaluasi CNN dilakukan dengan fungsi `evaluate_model()`, yang memuat model Keras, menjalankan prediksi pada data test, lalu menghitung `macro_f1`, confusion matrix, jumlah parameter, dan menyimpan prediksi ke CSV.

Evaluasi captioning dilakukan dengan BLEU-4 dan METEOR. Fungsi `bleu_score()` menghitung modified n-gram precision hingga 4-gram serta brevity penalty. Fungsi `meteor_score()` menghitung skor berdasarkan kecocokan token, precision, recall, fragmentasi, dan penalty. Fungsi `evaluate()` menghasilkan caption untuk setiap gambar menggunakan greedy decoding atau beam search, lalu menghitung rata-rata BLEU-4 dan METEOR.

2.2 Penjelasan Forward Propagation

Forward propagation adalah proses menghitung keluaran model dari input yang diberikan dengan melewati data secara berurutan melalui setiap layer. Pada tugas besar ini, forward propagation digunakan pada tiga jenis model utama, yaitu CNN untuk klasifikasi gambar, RNN untuk pemrosesan sequence caption, dan LSTM sebagai pengembangan RNN yang mampu menyimpan informasi jangka panjang.

2.2.1 Forward Propagation CNN

Pada CNN, input berupa citra RGB dengan bentuk umum (height, width, channels), misalnya (64, 64, 3). Citra ini diproses melalui beberapa layer konvolusi, pooling, flatten, dense, dan softmax.

Tahap pertama adalah convolution layer. Pada layer Conv2D, input gambar diproses menggunakan sejumlah kernel atau filter. Setiap filter digeser pada area spasial gambar. Untuk setiap posisi, diambil patch input dengan ukuran sama seperti kernel, kemudian dilakukan operasi perkalian elemen dan penjumlahan. Secara umum, operasi konvolusi dapat ditulis sebagai:

$$Y[i, j, k] = \text{sum}(X_{\text{patch}} * W_k) + b_k$$

dengan X_{patch} adalah bagian kecil dari input, W_k adalah kernel ke- k , dan b_k adalah bias untuk filter tersebut. Hasil dari operasi ini adalah feature map yang merepresentasikan pola visual tertentu, seperti tepi, tekstur, atau bentuk.

Setelah hasil linear diperoleh, fungsi aktivasi diterapkan. Pada implementasi ini, CNN menggunakan aktivasi ReLU:

$$\text{ReLU}(x) = \max(0, x)$$

ReLU membuat nilai negatif menjadi nol dan mempertahankan nilai positif. Aktivasi ini membantu model mempelajari hubungan non-linear pada gambar.

Pada model non-shared CNN, digunakan layer LocallyConnected2D. Forward propagation-nya mirip dengan Conv2D, tetapi setiap posisi spasial memiliki kernel yang berbeda. Dengan demikian, bobot tidak digunakan ulang di seluruh gambar. Operasi ini membuat jumlah parameter lebih besar, tetapi memungkinkan model mempelajari pola lokal yang spesifik terhadap posisi.

Setelah convolution atau locally connected layer, data masuk ke pooling layer. Pooling digunakan untuk mengurangi ukuran spasial feature map dan mempertahankan informasi penting. Pada MaxPooling2D, setiap region kecil diambil nilai maksimumnya:

$$Y[i, j, c] = \max(X_{\text{region}})$$

Sedangkan pada AveragePooling2D, setiap region diganti dengan rata-rata nilainya:

$$Y[i, j, c] = \text{mean}(X_{\text{region}})$$

Pooling membantu mengurangi kompleksitas komputasi dan membuat representasi lebih tahan terhadap sedikit pergeseran posisi objek.

Setelah beberapa blok convolution dan pooling, feature map diubah menjadi vektor satu dimensi menggunakan layer Flatten. Jika input flatten berbentuk (H, W, C) , maka output-nya menjadi:

$$H * W * C$$

Vektor hasil flatten kemudian masuk ke dense layer. Pada layer dense, setiap neuron terhubung ke seluruh elemen input. Operasi forward-nya adalah:

$$Z = XW + b$$

Kemudian hasilnya diberi aktivasi, misalnya ReLU untuk hidden layer.

Layer terakhir adalah dense layer dengan aktivasi softmax. Softmax mengubah skor mentah model menjadi probabilitas untuk setiap kelas:

$$\text{softmax}(z_i) = \exp(z_i) / \sum(\exp(z_j))$$

Output akhir CNN berupa vektor probabilitas dengan panjang sama dengan jumlah kelas. Kelas prediksi ditentukan dari indeks dengan probabilitas terbesar:

$$\text{predicted_class} = \text{argmax}(\text{probabilities})$$

Dengan demikian, alur forward propagation CNN adalah:

Input image → Convolution / Locally Connected → ReLU → Pooling → Flatten → Dense →
ReLU → Dense → Softmax → Class probabilities

2.2.2 Forward Propagation RNN

RNN digunakan untuk memproses data sequence, yaitu urutan token caption. Berbeda dari CNN yang memproses gambar secara spasial, RNN memproses input secara timestep demi timestep. Pada image captioning, input RNN berupa embedding kata dan fitur gambar yang sudah diproyeksikan.

Sebelum masuk ke RNN, token caption diubah menjadi vektor embedding menggunakan EmbeddingLayer. Jika token memiliki indeks t , maka embedding-nya diambil dari matriks embedding:

$$x_t = W_{\text{embedding}}[t]$$

Selain caption, model juga menerima fitur gambar dari CNN encoder. Fitur gambar tersebut diproyeksikan ke dimensi embedding menggunakan dense layer:

$$x_{\text{img}} = \text{image_feature} W_{\text{img}} + b_{\text{img}}$$

Pada metode injection pre, fitur gambar dianggap sebagai input awal sebelum token caption. Artinya, decoder terlebih dahulu menerima x_{img} , lalu memproses token-token caption. Urutannya menjadi:

$$[x_{\text{img}}, x_1, x_2, \dots, x_t]$$

Pada RNN sederhana, setiap timestep memiliki hidden state. Hidden state menyimpan informasi dari token-token sebelumnya. Persamaan forward propagation pada SimpleRNNCell adalah:

$$h_t = \tanh(x_t W_x + h_{t-1} W_h + b)$$

dengan:

- x_t adalah input pada timestep ke- t ,
- h_{t-1} adalah hidden state sebelumnya,
- W_x adalah bobot input ke hidden state,
- W_h adalah bobot hidden state sebelumnya ke hidden state saat ini,
- b adalah bias,
- h_t adalah hidden state baru.

Pada timestep pertama, hidden state biasanya diinisialisasi dengan nol:

$$h_0 = 0$$

Jika decoder memiliki beberapa layer, output hidden state dari layer pertama menjadi input untuk layer berikutnya. Untuk setiap timestep, prosesnya adalah:

$$\text{Layer 1: } h_t^{(1)} = \tanh(x_t W_x^{(1)} + h_{t-1}^{(1)} W_h^{(1)} + b^{(1)})$$

$$\text{Layer 2: } h_t^{(2)} = \tanh(h_t^{(1)} W_x^{(2)} + h_{t-1}^{(2)} W_h^{(2)} + b^{(2)})$$

Output dari layer terakhir kemudian masuk ke dense output layer untuk menghasilkan distribusi probabilitas kata berikutnya:

$$p_t = \text{softmax}(h_t W_o + b_o)$$

Token dengan probabilitas tertinggi dipilih sebagai prediksi kata berikutnya pada greedy decoding:

$$\text{next_token} = \text{argmax}(p_t)$$

Pada proses training, model memproses seluruh sequence caption sekaligus dan menghasilkan probabilitas kata untuk setiap timestep. Pada proses inference, model menghasilkan kata satu per satu secara autoregresif. Kata yang diprediksi pada timestep sebelumnya akan digunakan sebagai input untuk timestep berikutnya.

Alur forward propagation RNN untuk captioning adalah:

Image feature → Image projection → Initial visual input / context
Caption token → Embedding → RNN cell per timestep → Dense output → Softmax → Next word probability

Kelemahan RNN sederhana adalah hidden state harus membawa seluruh informasi masa lalu dalam satu vektor. Pada sequence panjang, informasi dari timestep awal dapat melemah sehingga RNN rentan terhadap masalah vanishing gradient dan kurang efektif menyimpan dependensi jangka panjang.

2.2.3 Forward Propagation LSTM

LSTM digunakan untuk mengatasi keterbatasan RNN dalam menyimpan informasi jangka panjang. Perbedaan utama LSTM adalah adanya dua state, yaitu hidden state h_t dan cell state c_t .

Hidden state digunakan sebagai output pada timestep saat ini, sedangkan cell state berfungsi sebagai memori jangka panjang. LSTM mengatur informasi yang disimpan, dibuang, dan dikeluarkan melalui beberapa gate.

Pada implementasi LSTMCell, input x_t dan hidden state sebelumnya h_{t-1} terlebih dahulu digabungkan secara linear:

$$z = x_t W_x + h_{t-1} W_h + b$$

Vektor z kemudian dibagi menjadi empat bagian untuk menghitung empat komponen utama LSTM:

- $i_t = \text{sigmoid}(z_i)$
- $f_t = \text{sigmoid}(z_f)$
- $g_t = \tanh(z_g)$
- $o_t = \text{sigmoid}(z_o)$

Masing-masing komponen memiliki fungsi berbeda.

- i_t adalah input gate, yang menentukan seberapa banyak informasi baru akan dimasukkan ke cell state.
- f_t adalah forget gate, yang menentukan seberapa banyak informasi lama dari cell state sebelumnya akan dipertahankan.
- g_t adalah candidate value, yaitu kandidat informasi baru yang dapat dimasukkan ke memori.

- o_t adalah output gate, yang menentukan seberapa banyak isi cell state akan dikeluarkan sebagai hidden state.

Cell state baru dihitung dengan:

$$c_t = f_t * c_{t-1} + i_t * g_t$$

Bagian $f_t * c_{t-1}$ mengontrol memori lama yang dipertahankan, sedangkan $i_t * g_t$ mengontrol informasi baru yang ditambahkan.

Hidden state baru dihitung dengan:

$$h_t = o_t * \tanh(c_t)$$

Dengan mekanisme ini, LSTM dapat mempertahankan informasi penting lebih lama dibandingkan RNN sederhana. Jika suatu informasi masih relevan, forget gate dapat mempertahankannya di cell state. Jika informasi baru penting, input gate dapat memasukkannya ke memori.

Pada decoder LSTM berlapis, output hidden state dari layer pertama menjadi input untuk layer berikutnya. Setiap layer memiliki hidden state dan cell state sendiri:

Layer 1: $h_t^{(1)}, c_t^{(1)}$

Layer 2: $h_t^{(2)}, c_t^{(2)}$

...

Layer n: $h_t^{(n)}, c_t^{(n)}$

Output dari layer LSTM terakhir diteruskan ke dense layer softmax untuk menghasilkan probabilitas kata berikutnya:

$$p_t = \text{softmax}(h_t W_o + b_o)$$

Pada image captioning, LSTM menerima fitur gambar dengan cara yang sama seperti RNN. Pada metode pre, fitur gambar diproses sebagai token awal untuk membentuk state awal decoder. Setelah itu, token <start> diberikan, lalu model mulai memprediksi kata-kata caption sampai menghasilkan token <end> atau mencapai panjang maksimum.

Alur forward propagation LSTM adalah:

Image feature → Image projection → LSTM initial context

Caption token → Embedding → LSTM gates → Cell state update → Hidden state update →
Dense output → Softmax → Next word probability

Jika dibandingkan dengan RNN sederhana, LSTM memiliki proses forward propagation yang lebih kompleks karena melibatkan beberapa gate. Namun, kompleksitas tersebut membuat LSTM lebih cocok untuk caption generation karena caption membutuhkan pemahaman urutan kata dan konteks gambar secara berkelanjutan.

2.3 Implementasi Bonus

2.3.1 Visualisasi Fitur CNN

Bonus visualisasi fitur diimplementasikan untuk membantu interpretasi model CNN setelah proses klasifikasi. Visualisasi dilakukan dalam dua bentuk, yaitu intermediate feature maps dan Grad-CAM.

Intermediate feature maps digunakan untuk melihat respons aktivasi dari suatu layer konvolusi terhadap sebuah gambar input. Pada implementasi ini, model Keras yang sudah dilatih dimuat kembali, lalu dibuat model tambahan dengan input yang sama tetapi output-nya berasal dari layer konvolusi tertentu. Jika nama layer tidak diberikan, sistem otomatis memilih layer Conv2D terakhir. Output feature map kemudian divisualisasikan dalam bentuk grid, dengan setiap kanal dinormalisasi ke rentang 0 sampai 1 agar pola aktivasi lebih mudah diamati.

Selain itu, diimplementasikan juga Grad-CAM untuk menunjukkan bagian gambar yang paling berpengaruh terhadap prediksi kelas tertentu. Grad-CAM dihitung dengan mengambil gradien skor kelas terhadap output layer konvolusi terakhir. Gradien tersebut dirata-ratakan secara spasial untuk memperoleh bobot tiap kanal feature map. Selanjutnya, feature map dikombinasikan menggunakan bobot tersebut, diberi fungsi ReLU, dinormalisasi, lalu di-overlay ke gambar asli sebagai heatmap.

Implementasi ini menghasilkan tiga keluaran utama, yaitu `feature_maps.png`, `grad_cam_heatmap.png`, dan `grad_cam_overlay.png`. Dengan visualisasi ini, prediksi model tidak hanya dievaluasi dari metrik seperti macro F1-score, tetapi juga dapat dianalisis secara kualitatif berdasarkan region gambar yang menjadi perhatian model.

2.3.2 Beam Search Decoder

Selain greedy decoding, diimplementasikan juga beam search decoder sebagai strategi decoding alternatif untuk image captioning. Greedy decoding memilih token dengan

probabilitas tertinggi pada setiap timestep, sehingga keputusan yang dibuat bersifat lokal. Sebaliknya, beam search menyimpan beberapa kandidat caption terbaik selama proses decoding.

Pada implementasi ini, beam search menggunakan ukuran beam $k = 3$ dan $k = 5$. Setiap kandidat caption menyimpan urutan token, skor log-probability kumulatif, serta state decoder. Untuk LSTM, state yang disimpan mencakup hidden state dan cell state. Pada setiap timestep, setiap kandidat diperluas menggunakan token-token dengan probabilitas tertinggi, lalu kandidat terbaik dipilih kembali berdasarkan skor yang sudah dinormalisasi dengan length penalty.

Hasil evaluasi menunjukkan bahwa beam search meningkatkan eksplorasi kandidat caption, tetapi tidak selalu meningkatkan BLEU-4 pada data evaluasi kecil. Pada RNN, beam size 5 menghasilkan BLEU-4 lebih tinggi dibanding greedy, tetapi METEOR cenderung menurun. Pada LSTM, greedy decoding masih memberikan skor yang lebih stabil dibanding beam search. Hal ini dapat terjadi karena model decoder belum cukup kuat, sehingga pencarian kandidat yang lebih luas tetap dapat memilih caption yang secara probabilistik tinggi tetapi kurang sesuai dengan ground truth.

Dengan demikian, beam search berhasil diimplementasikan sebagai alternatif decoding, tetapi kualitas akhirnya tetap sangat bergantung pada kualitas distribusi probabilitas yang dihasilkan decoder.

2.3.3 Image Captioning dengan Init-Inject

Pada spesifikasi utama, arsitektur image captioning menggunakan metode pre-inject, yaitu feature vector CNN diproyeksikan ke dimensi embedding lalu dimasukkan sebagai input timestep awal sebelum token <start>. Sebagai bonus, diimplementasikan arsitektur alternatif init-inject.

Pada init-inject, caption prefix diproses terlebih dahulu oleh decoder RNN atau LSTM. Setelah decoder menghasilkan output hidden state, representasi gambar yang sudah diproyeksikan digabungkan dengan output decoder. Penggabungan dilakukan menggunakan operasi Concatenate, sehingga dense output layer menerima gabungan informasi visual dan konteks teks. Dengan pendekatan ini, informasi gambar tidak diperlakukan sebagai token awal, tetapi sebagai konteks tambahan pada setiap prediksi token.

Berdasarkan hasil evaluasi pada 50 sampel, init-inject memberikan peningkatan performa dibandingkan pre-inject. Pada decoder RNN, BLEU-4 meningkat dari sekitar 0.0126 menjadi 0.0620, sedangkan METEOR meningkat dari sekitar 0.2370 menjadi 0.2996. Pada decoder

LSTM, BLEU-4 meningkat dari sekitar 0.0101 menjadi 0.0321, dan METEOR meningkat dari sekitar 0.2443 menjadi 0.2815.

Hasil ini menunjukkan bahwa penggabungan feature gambar setelah pemrosesan caption prefix dapat membantu model memanfaatkan konteks visual secara lebih langsung saat menghasilkan token berikutnya. Namun, waktu inferensi init-inject sedikit lebih besar, terutama pada RNN, karena ukuran representasi yang masuk ke output layer menjadi lebih besar akibat proses konkatenasi.

2.3.4 Batch Inference

Bonus batch inference diimplementasikan agar forward propagation from scratch dapat menerima lebih dari satu input dalam satu kali proses inferensi. Pada implementasi CNN, layer-layer scratch seperti Conv2D, LocallyConnected2D, pooling, global pooling, flatten, dan dense mendukung input dengan dimensi batch. Input tunggal tetap didukung, tetapi akan diubah sementara menjadi batch berukuran satu agar alur komputasi konsisten.

Pada bagian RNN/LSTM, batch inference diterapkan melalui fungsi `forward_sequence_batch`. Fungsi ini menerima kumpulan feature vector gambar dan caption token dalam bentuk batch. Feature gambar diproyeksikan secara batch, embedding caption juga dihitung secara batch, lalu seluruh sequence diproses oleh decoder. Output decoder kemudian diratakan sementara sebelum masuk ke dense output layer, lalu dikembalikan ke bentuk (batch, seq_len, vocab_size).

Hasil pengujian menunjukkan bahwa batch inference menghasilkan prediksi yang konsisten dengan inferensi satu per satu. Match rate untuk RNN dan LSTM tercatat 1.0, yang berarti hasil batch sama dengan hasil single inference. Dari sisi waktu, batch inference pada LSTM lebih efisien, dengan rata-rata waktu turun dari sekitar 3.28 ms per sampel menjadi 1.36 ms per sampel. Pada RNN, waktu batch sedikit lebih besar pada pengujian kecil, tetapi hasil tetap benar dan konsisten.

2.3.5 Backward Propagation from Scratch

Bonus backward propagation diimplementasikan untuk layer-layer utama pada CNN scratch. Setiap layer menyimpan nilai input dan pre-activation saat forward propagation agar dapat digunakan kembali pada proses backward. Gradien dihitung terhadap input layer sebelumnya serta terhadap parameter layer seperti kernel dan bias.

Layer yang sudah memiliki fungsi backward meliputi Conv2D, LocallyConnected2D, MaxPooling2D, AveragePooling2D, GlobalAveragePooling2D, GlobalMaxPooling2D, Flatten, dan Dense. Fungsi aktivasi seperti ReLU dan softmax juga dilengkapi turunan backward. Selain

itu, loss cross-entropy menyediakan gradien terhadap logits atau probabilitas sehingga dapat menjadi titik awal backpropagation.

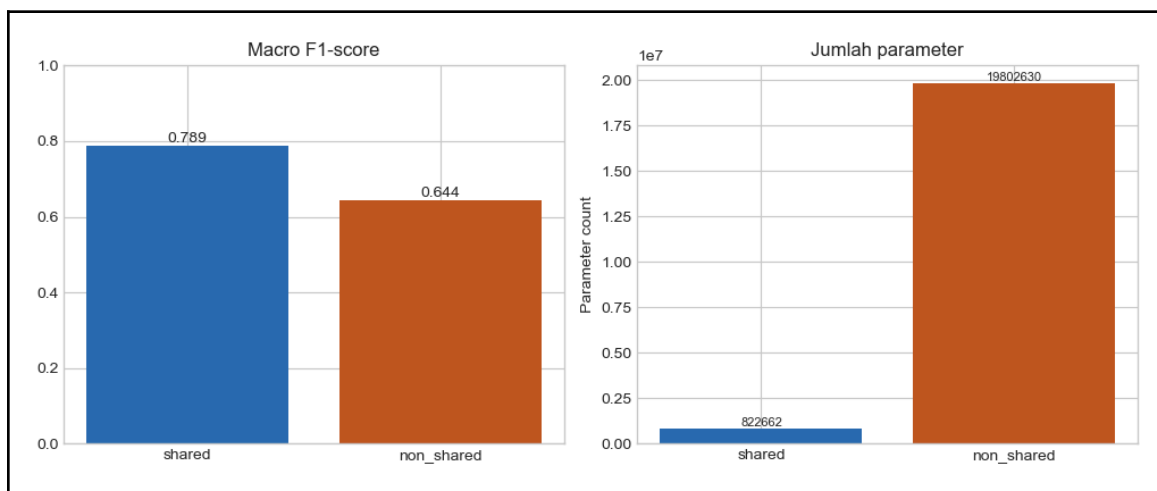
Untuk memvalidasi implementasi, dilakukan sanity check menggunakan numerical gradient. Pada pengujian CNN, relative error gradien kernel Dense sekitar 0.0130, sedangkan relative error gradien kernel Conv2D sekitar 0.00436. Nilai error yang kecil menunjukkan bahwa arah dan besaran gradien hasil implementasi backward sudah cukup dekat dengan pendekatan numerik.

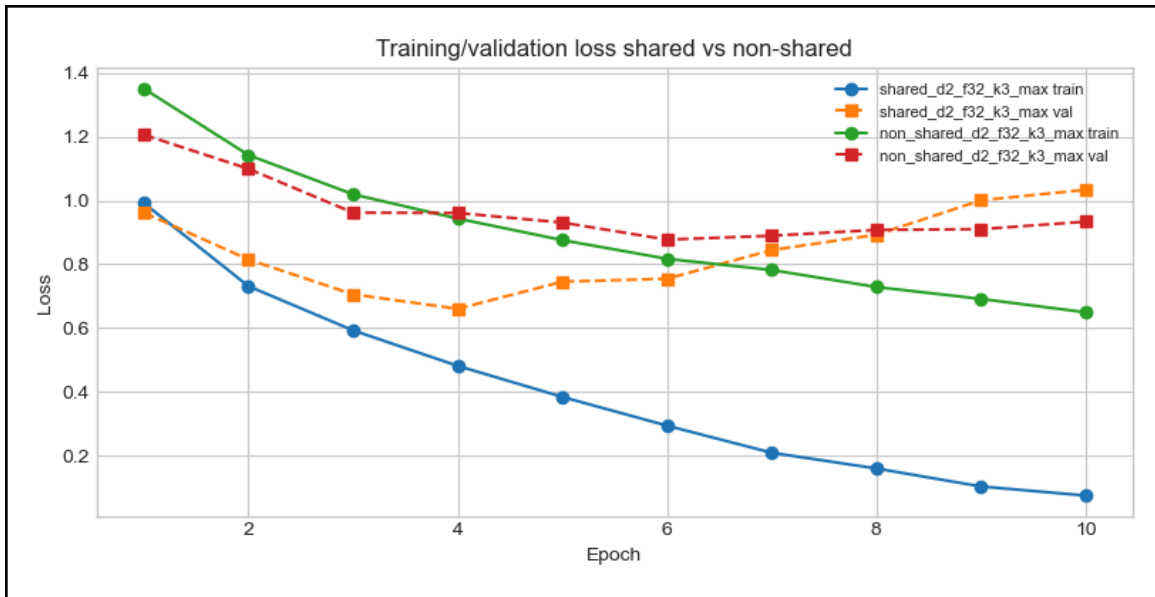
Implementasi backward propagation ini menunjukkan bahwa model scratch tidak hanya dapat melakukan inferensi, tetapi juga memiliki fondasi untuk proses pelatihan manual dari awal menggunakan NumPy.

2.4 Hasil Pengujian

2.4.1 CNN

a. Perbandingan Shared vs Non-Shared Parameter

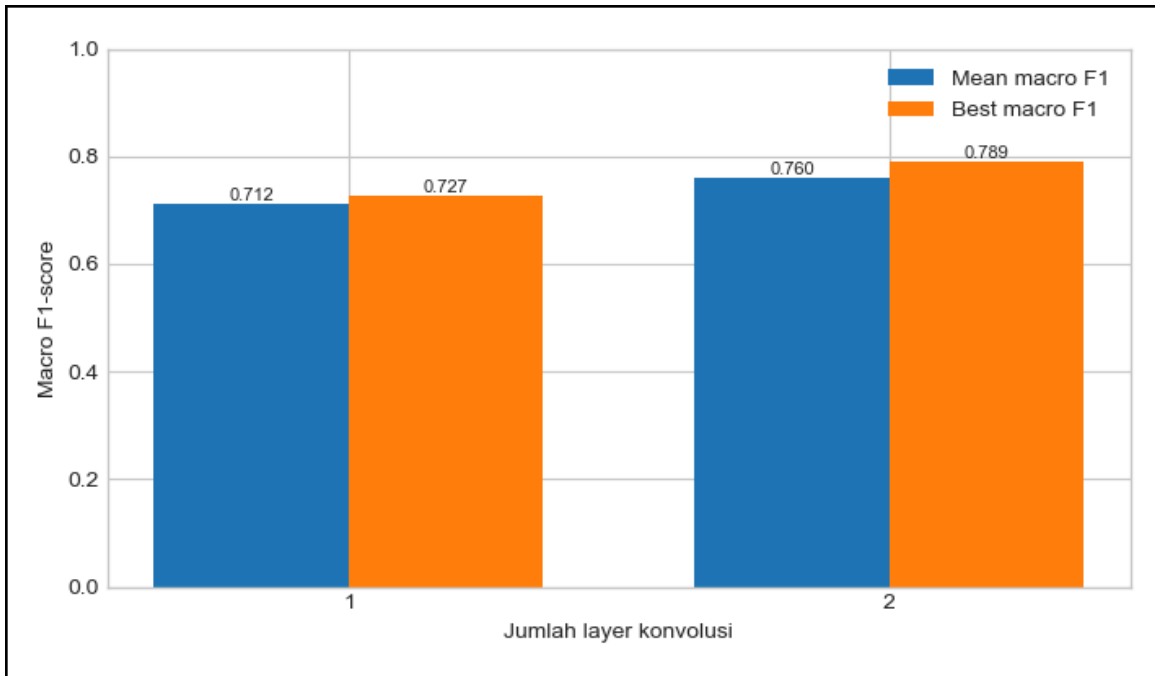




Analisis:

Model shared terbaik (shared_d2_f32_k3_max) memperoleh macro F1 sekitar 0.789 dengan 822.662 parameter, sedangkan model non-shared (non_shared_d2_f32_k3_max) memperoleh macro F1 sekitar 0.644 dengan 19.802.630 parameter. Non-shared memiliki parameter jauh lebih banyak karena setiap posisi spasial memiliki bobot sendiri, berbeda dari Conv2D shared yang memakai kernel sama di seluruh posisi. Hasil ini menunjukkan bahwa parameter sharing membuat model jauh lebih efisien dan pada eksperimen ini juga lebih akurat. Non-shared meningkatkan kapasitas model secara besar, tetapi kapasitas tersebut tidak otomatis meningkatkan generalisasi, yang mana model menjadi lebih berat dan lebih mudah overfit atau sulit dioptimasi.

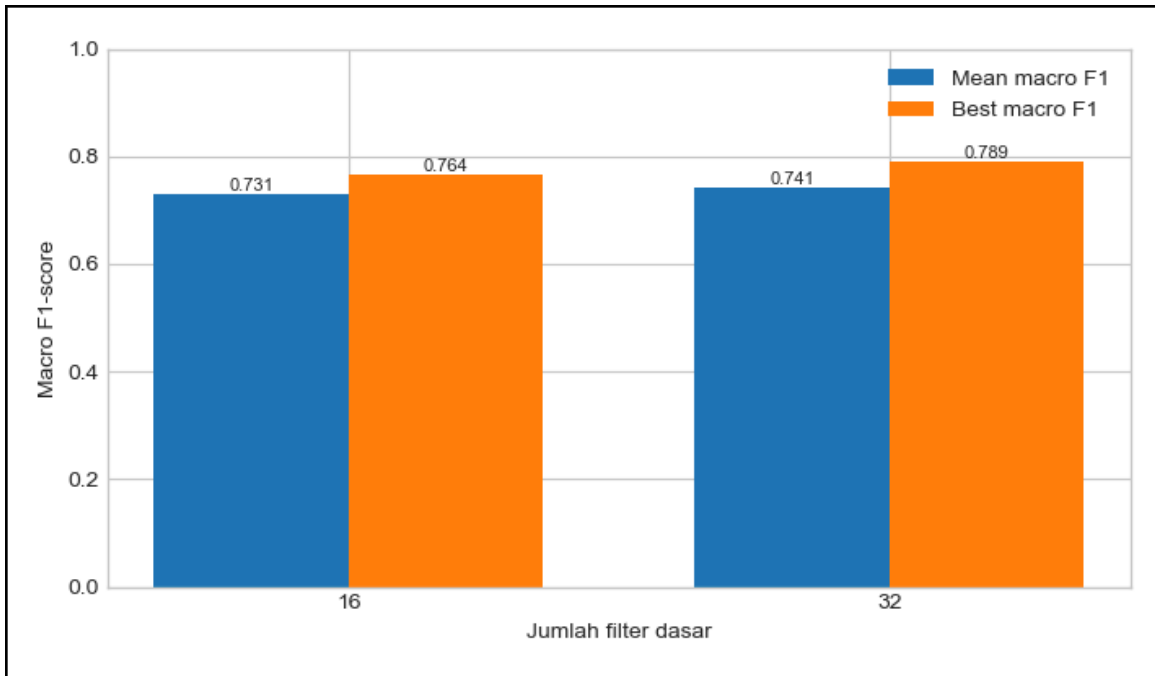
b. Pengaruh jumlah layer konvolusi



Analisis:

Model dengan 2 layer konvolusi memberikan performa lebih baik dibanding 1 layer konvolusi. Rata-rata macro F1 untuk 2 layer adalah sekitar 0.760, sedangkan 1 layer sekitar 0.712. Hasil ini menunjukkan bahwa penambahan kedalaman konvolusi membantu model menangkap pola visual yang lebih kompleks, misalnya kombinasi edge, tekstur, dan struktur objek. Selain performa, konfigurasi 2 layer pada eksperimen ini juga memiliki rata-rata jumlah parameter yang lebih kecil dibanding 1 layer karena feature map diperkecil lebih awal oleh pooling sebelum masuk ke Dense layer. Dengan demikian, 2 layer konvolusi memberi representasi yang lebih baik sekaligus lebih efisien pada konfigurasi yang diuji.

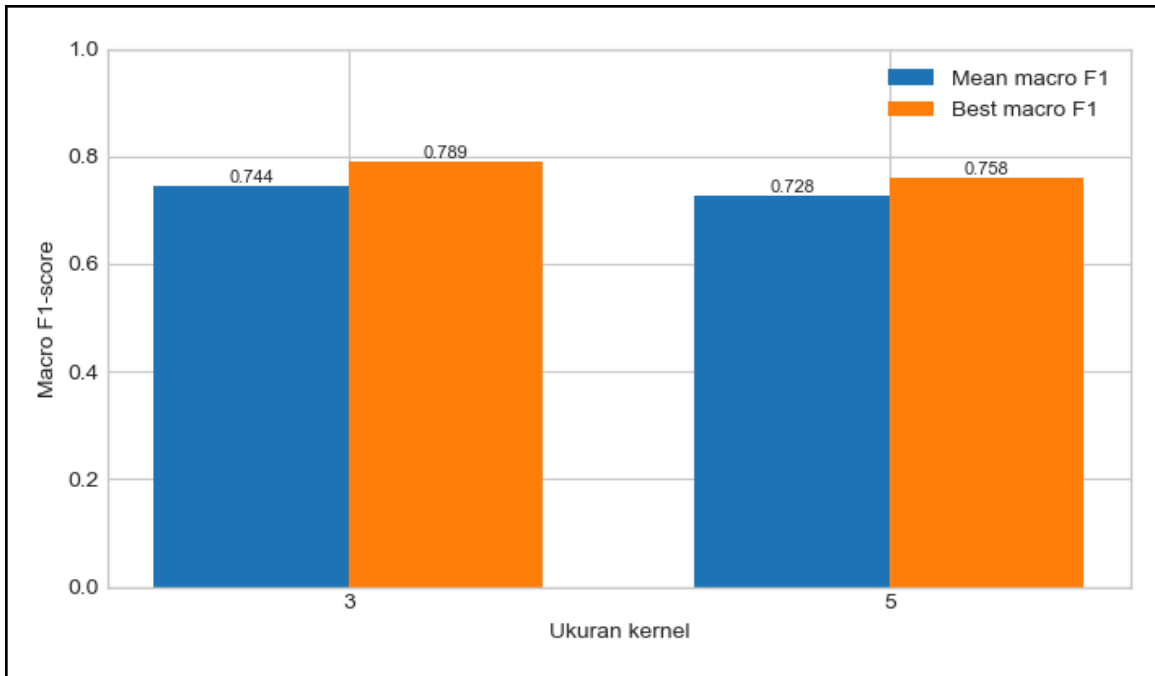
c. Pengaruh banyak filter per layer konvolusi



Analisis:

Peningkatan filter dari 16 ke 32 menaikkan performa dengan cukup baik. Rata-rata macro F1 untuk filter 32 sekitar 0.741, sedikit lebih tinggi dibanding filter 16 sekitar 0.731. Model terbaik keseluruhan juga menggunakan filter 32. Hal ini masuk akal karena jumlah filter yang lebih besar meningkatkan kapasitas model untuk mempelajari variasi fitur visual pada enam kelas Intel Image Classification. Namun, kenaikannya tidak terlalu besar dan dibayar dengan jumlah parameter yang lebih tinggi. Artinya, filter lebih banyak dapat membantu performa, tetapi trade-off terhadap ukuran model dan kebutuhan komputasi tetap perlu diperhatikan.

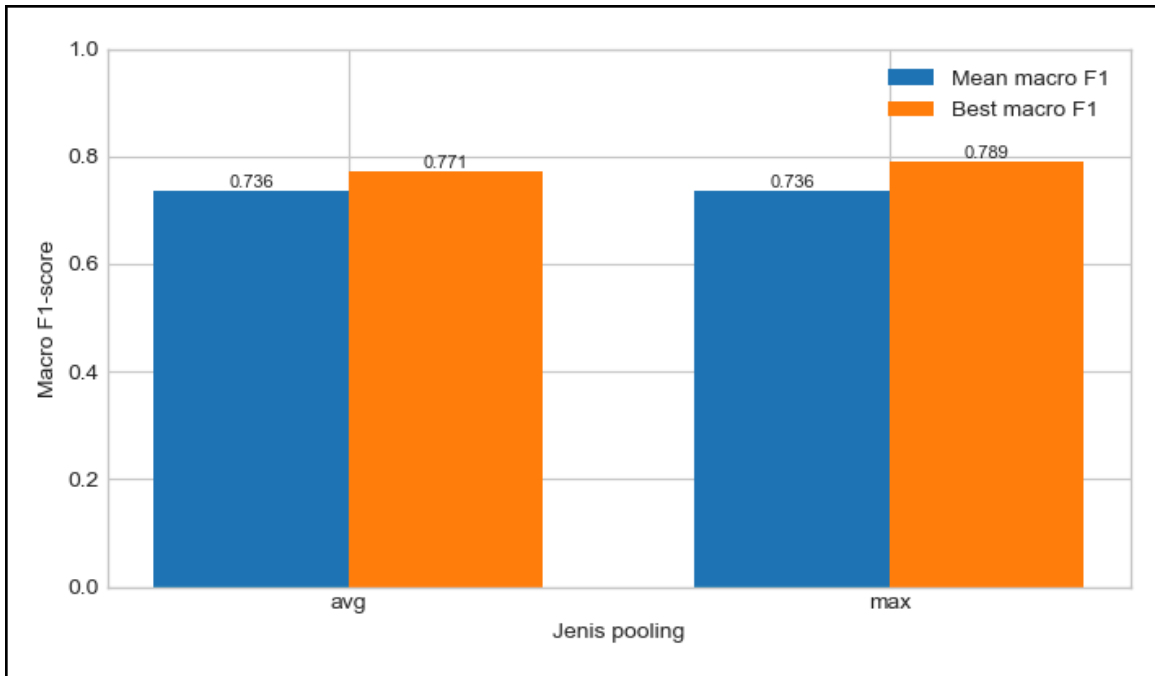
d. Pengaruh ukuran filter per layer konvolusi



Analisis:

Kernel 3x3 menghasilkan performa lebih baik dibanding kernel 5x5 pada eksperimen ini. Rata-rata macro F1 kernel 3x3 sekitar 0.744, sedangkan kernel 5x5 sekitar 0.728. Best macro F1 juga diperoleh dari kernel 3x3. Kernel 3x3 cenderung lebih efektif karena menangkap pola lokal yang lebih halus dan menjaga lebih banyak detail spasial sebelum pooling. Kernel 5x5 memiliki receptive field lebih besar pada satu layer, tetapi dapat mengurangi resolusi feature map lebih cepat pada padding valid dan berpotensi kehilangan detail lokal. Untuk dataset scene classification seperti Intel Image Classification, pola lokal seperti tekstur bangunan, vegetasi, laut, atau jalan tampaknya lebih efektif ditangkap dengan kernel 3x3.

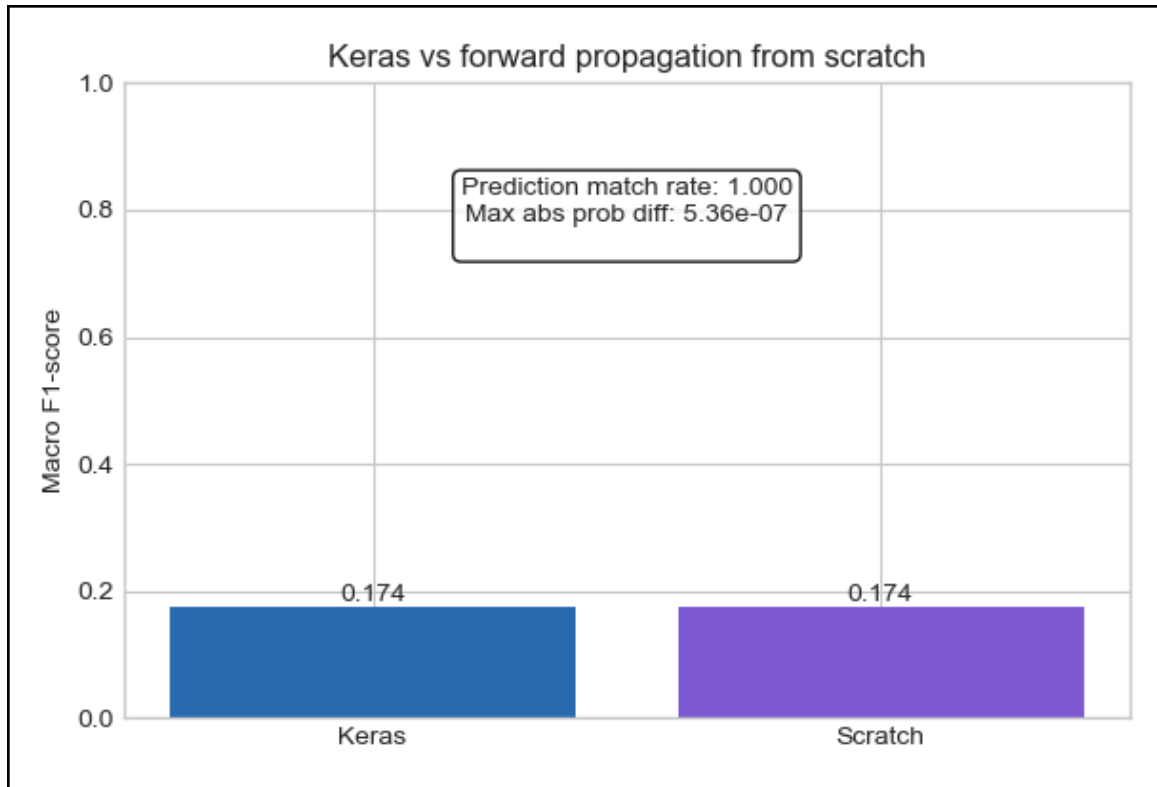
e. Pengaruh jenis pooling layer



Analisis:

Perbedaan rata-rata performa antara max pooling dan average pooling sangat kecil. Rata-rata macro F1 max pooling sekitar 0.736, sedangkan average pooling sekitar 0.736 juga, dengan max pooling sedikit lebih tinggi pada konfigurasi terbaik. Max pooling unggul pada model terbaik (shared_d2_f32_k3_max), kemungkinan karena max pooling mempertahankan aktivasi fitur paling kuat sehingga cocok untuk mendeteksi pola diskriminatif seperti tepi bangunan, tekstur hutan, atau garis jalan. Average pooling tetap kompetitif karena merangkum informasi area secara lebih halus. Dari hasil ini, jenis pooling bukan faktor paling dominan dibanding jumlah layer atau ukuran kernel, tetapi max pooling menjadi pilihan terbaik pada konfigurasi final.

f. Perbandingan Keras vs from scratch



Analisis:

Forward propagation from scratch menghasilkan macro F1 yang sama dengan Keras pada sampel pengujian yang digunakan, yaitu sekitar 0.174 untuk 30 sampel, dengan prediction match rate 1.000 dan perbedaan probabilitas maksimum sekitar $5.36e-07$. Nilai macro F1 pada perbandingan ini kecil karena evaluasi scratch dibatasi pada subset kecil ($\text{max_samples}=30$), bukan seluruh test set. Fokus utama perbandingan ini adalah kesesuaian implementasi forward propagation, bukan performa final model. Prediction match rate 100% dan selisih probabilitas yang sangat kecil menunjukkan bahwa implementasi scratch berhasil mereplikasi urutan operasi Keras untuk arsitektur shared terbaik, termasuk Conv2D, pooling, Flatten, Dense, ReLU, dan Softmax.

g. Kesimpulan Pengujian CNN

Berdasarkan 16 eksperimen CNN shared parameter, konfigurasi terbaik diperoleh pada arsitektur dengan 2 layer konvolusi, filter dasar 32, kernel 3x3, dan pooling terbaik sesuai hasil summary.csv. Secara umum, model dengan 2 layer konvolusi memberikan performa lebih baik dibanding 1 layer karena mampu mempelajari fitur visual yang lebih hierarkis. Penambahan jumlah filter juga cenderung meningkatkan performa, meskipun disertai kenaikan jumlah parameter dan biaya komputasi.

Ukuran kernel 3x3 menghasilkan performa lebih baik dibanding 5x5. Hal ini menunjukkan bahwa fitur lokal yang lebih kecil dan detail lebih efektif untuk task klasifikasi scene pada Intel Image Classification. Sementara itu, perbedaan antara max pooling dan average pooling relatif kecil, sehingga jenis pooling bukan faktor paling dominan. Namun, konfigurasi terbaik tetap menunjukkan bahwa pooling tertentu dapat memberi sedikit keuntungan tergantung kombinasi hyperparameter lainnya.

Perbandingan shared dan non-shared parameter menunjukkan bahwa CNN dengan shared parameter jauh lebih efisien. Model non-shared memiliki jumlah parameter yang jauh lebih besar karena setiap posisi spasial memiliki bobot sendiri, tetapi performanya tidak lebih baik dari model shared. Ini menunjukkan bahwa parameter sharing pada Conv2D penting untuk efisiensi dan generalisasi, terutama karena fitur visual pada gambar dapat muncul di berbagai posisi.

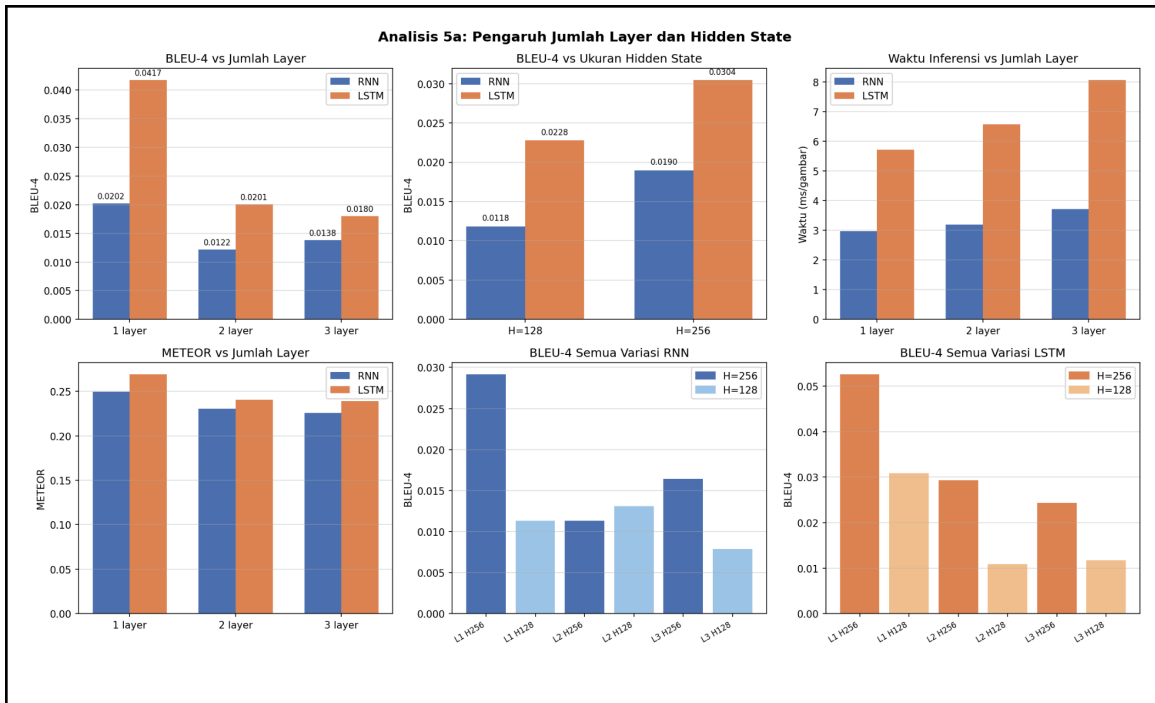
Implementasi forward propagation from scratch juga berhasil divalidasi terhadap model Keras. Hasil prediksi scratch sama dengan Keras pada sampel uji yang digunakan, dengan prediction match rate 1.0 dan selisih probabilitas yang sangat kecil. Ini menunjukkan bahwa implementasi layer-layer CNN from scratch, proses load bobot, dan urutan operasi forward propagation sudah konsisten dengan model Keras.

Secara keseluruhan, pengujian menunjukkan bahwa arsitektur CNN shared parameter dengan depth lebih besar, jumlah filter lebih banyak, kernel 3x3, dan konfigurasi pooling yang tepat memberikan performa terbaik untuk klasifikasi citra. CNN shared parameter menjadi pilihan utama karena memberikan keseimbangan terbaik antara performa, jumlah parameter, efisiensi komputasi, dan kemampuan generalisasi.

2.4.2 Simple RNN dan LSTM untuk Image Captioning

a. Perbandingan Jumlah Layer dan Hidden State

i. BLEU-4 dan METEOR



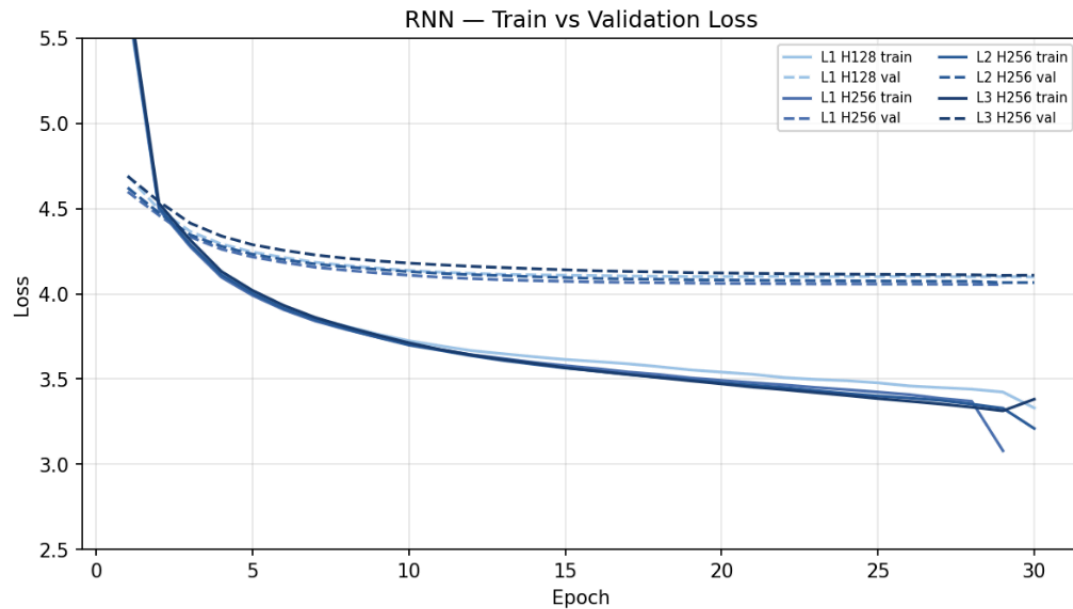
Analisis:

Pola LSTM lebih terstruktur dibanding RNN. Terdapat perbedaan yang jelas H256 > H128 di setiap jumlah layer. L1 H256 (0.0526) unggul dengan besar, hampir 2x di atas L1 H128 (0.0308). LSTM dengan lebih banyak layer (L2, L3) namun hidden besar masih menghasilkan skor yang layak (0.029-0.024), berbeda dengan RNN di mana layer tambahan langsung merusak performa.

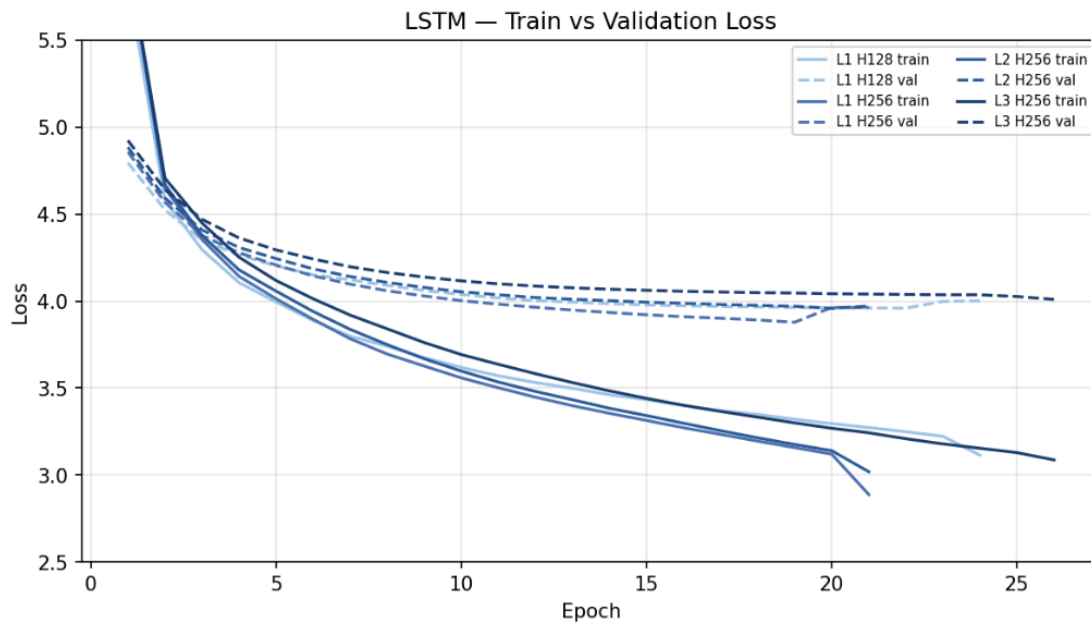
L2 H128 (0.0108) lebih buruk dari L3 H128 (0.0117). Ini kemungkinan karena randomness pada training kedua model memiliki val_loss sangat mirip (3.9931 vs 4.0868) dan skor BLEU yang sama-sama sangat rendah, sehingga variasi kecil dalam parameter akhir memberikan efek besar pada 4-gram precision.

ii. Grafik Training dan Validation Loss

Training Loss & Validation



on Loss (RNN vs LSTM)



Analisis:

Grafik menunjukkan dua pola utama yang konsisten pada kedua arsitektur. Konfigurasi H=128 pada RNN maupun LSTM konvergen cepat namun plateau di nilai loss tinggi sekitar 4.0 hingga 4.1 sejak epoch 5, mengindikasikan underfitting akibat kapasitas model yang terlalu kecil. Konfigurasi H=256 menunjukkan training loss yang terus turun hingga akhir pelatihan, namun validation loss tetap plateau lebih

awal dan membentuk gap yang menandakan mild overfitting.

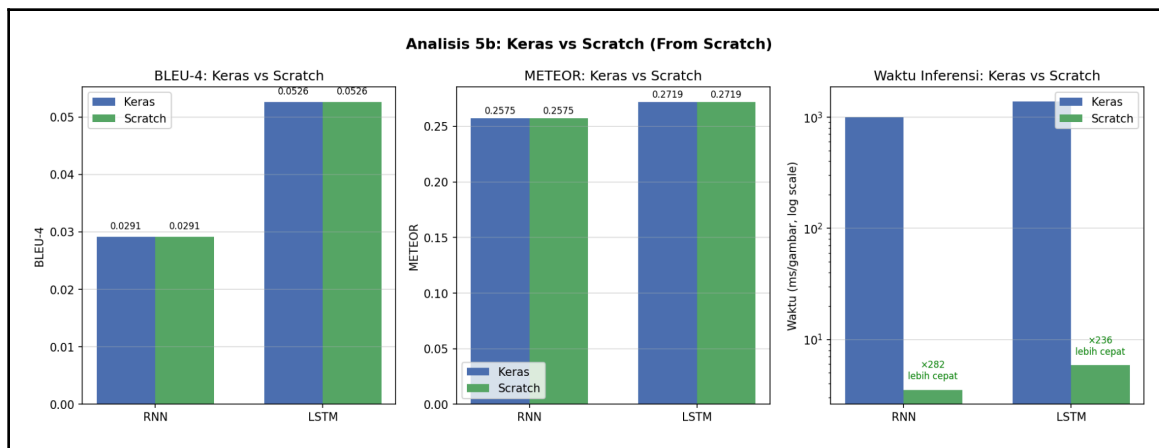
Penambahan layer dari L1 ke L2 dan L3 tidak memberikan perbaikan pada validation loss di kedua arsitektur, bahkan cenderung memperburuk konvergensi akibat vanishing gradient yang semakin parah pada jaringan lebih dalam. LSTM secara keseluruhan menghasilkan validation loss akhir yang lebih rendah dibanding RNN pada konfigurasi setara, dengan L1 H256 LSTM mencapai sekitar 3.92 berbanding 4.05 pada RNN. Hal ini mengonfirmasi bahwa konfigurasi optimal adalah L1 H256, dan memperbesar hidden state jauh lebih efektif dibanding menambah jumlah layer.

iii. Kesimpulan

Konfigurasi optimal untuk kedua arsitektur adalah 1 layer dengan hidden state 256. Penambahan layer tidak memberikan manfaat dan justru menurunkan kualitas caption akibat vanishing gradient yang semakin parah pada jaringan lebih dalam, sementara validation loss tidak ikut membaik meski training loss terus turun. Memperbesar hidden state dari 128 ke 256 terbukti jauh lebih efektif dibanding menambah jumlah layer. LSTM secara konsisten menghasilkan BLEU-4 dan METEOR lebih tinggi dari RNN di semua konfigurasi, dengan trade-off waktu inferensi sekitar 2 kali lebih lama.

b. Perbandingan Keras vs Scratch

i. Score dan Waktu Eksekusi

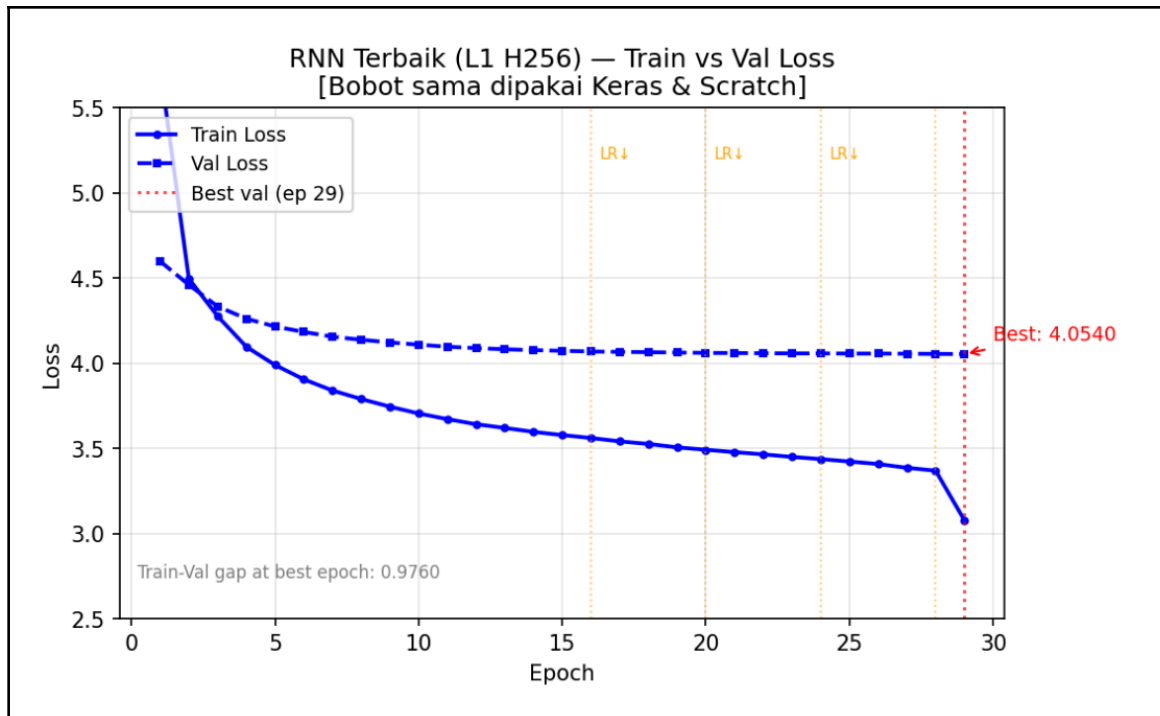


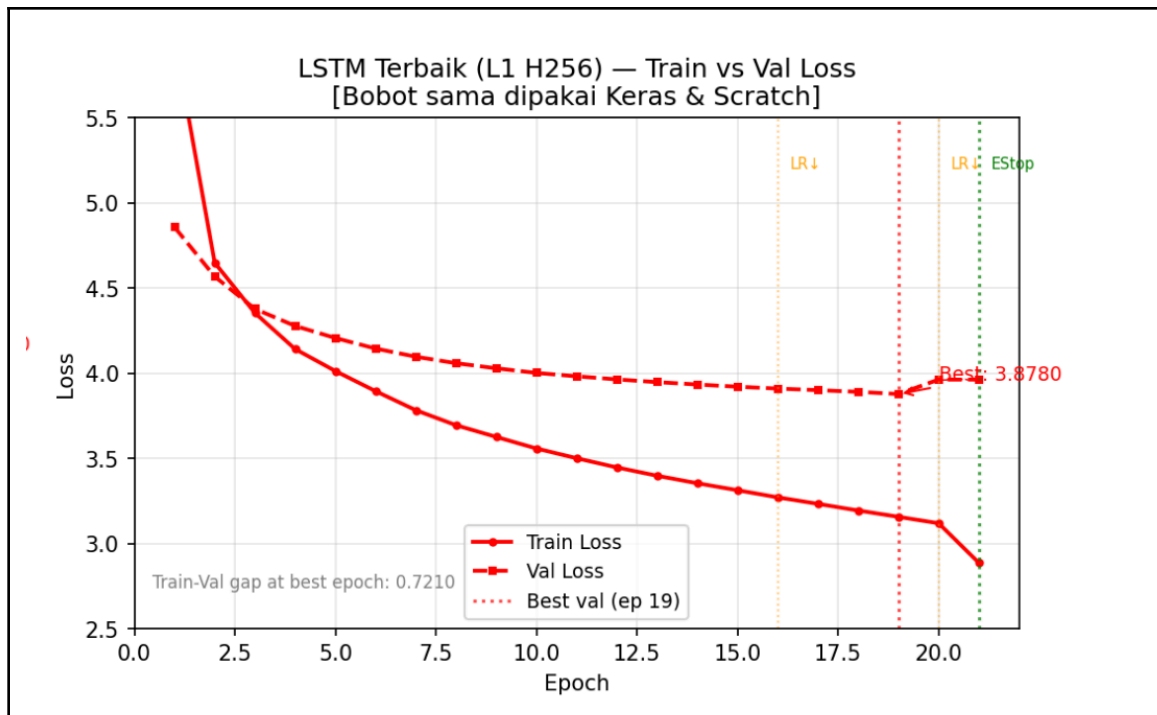
Analisis:

Dari segi score, implementasi Keras dan from scratch menghasilkan nilai yang identik pada kedua arsitektur. RNN Keras maupun Scratch sama-sama mencapai BLEU-4 0.0291 dan METEOR 0.2575, sedangkan LSTM Keras maupun Scratch keduanya menghasilkan BLEU-4 0.0526 dan METEOR 0.2719. Kesamaan ini membuktikan bahwa implementasi from scratch secara matematis ekuivalen dengan Keras dan tidak ada perbedaan kapasitas model maupun proses pelatihan di antara keduanya.

Perbedaan signifikan muncul pada waktu inferensi. Scratch jauh lebih cepat karena Keras memanggil `model.predict()` sebanyak 35 kali per gambar (1 call per token step), dengan overhead Python $\rightarrow$ TensorFlow dispatch sekitar 30ms per call. Scratch menjalankan matrix multiply NumPy kecil (sekitar 0.1ms/step) tanpa overhead tersebut. Ini bukan kelemahan Keras, melainkan trade-off karena Keras dirancang untuk training (batch besar, GPU).

ii. Training dan Validation Loss





Analisis:

Karena Keras dan from scratch menggunakan bobot yang sama, kurva training dan validation loss yang ditampilkan berlaku identik untuk kedua implementasi.

Pada RNN L1 H256, training loss turun secara konsisten dari sekitar 4.6 di epoch 1 hingga mencapai 3.1 di epoch 30. Namun validation loss berhenti menurun sejak epoch 5 dan plateau di kisaran 4.05 hingga akhir pelatihan, dengan best validation loss tercatat 4.0540 di epoch 29. Gap antara training dan validation loss di best epoch mencapai 0.9760, yang mengindikasikan overfitting cukup signifikan. Learning rate diturunkan beberapa kali (terlihat dari marker LR di grafik) namun tidak berhasil mendorong penurunan validation loss lebih lanjut.

Pada LSTM L1 H256, training loss turun lebih cepat dan validation loss ikut menurun secara lebih konsisten hingga sekitar epoch 19, di mana best validation loss tercatat 3.8780. Early stopping aktif di epoch 21 karena tidak ada perbaikan lebih lanjut. Gap train-val di best epoch hanya 0.7210, lebih kecil dibanding RNN, menandakan LSTM menggeneralisasi lebih baik. Secara keseluruhan, LSTM menunjukkan proses pelatihan yang lebih sehat dengan validation loss lebih rendah dan gap overfitting yang lebih kecil dibanding RNN.

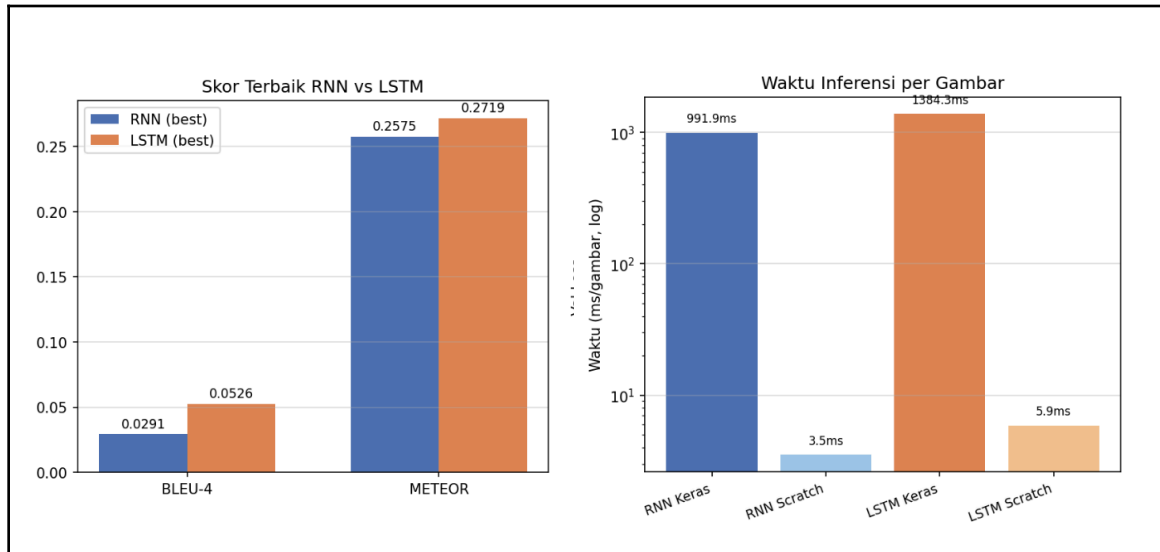
iii. Kesimpulan

Keras dan implementasi from scratch menghasilkan performa yang identik, dengan nilai BLEU-4 dan METEOR sama persis pada kedua arsitektur. Kurva training dan validation loss keduanya juga overlapping sempurna karena

menggunakan bobot yang sama. Perbedaan satu-satunya terletak pada waktu inferensi. Keras lebih cocok untuk tahap eksperimen, sedangkan implementasi from scratch lebih unggul untuk kebutuhan inferensi cepat.

c. Perbandingan RNN vs LSTM

i. Score dan Waktu Eksekusi

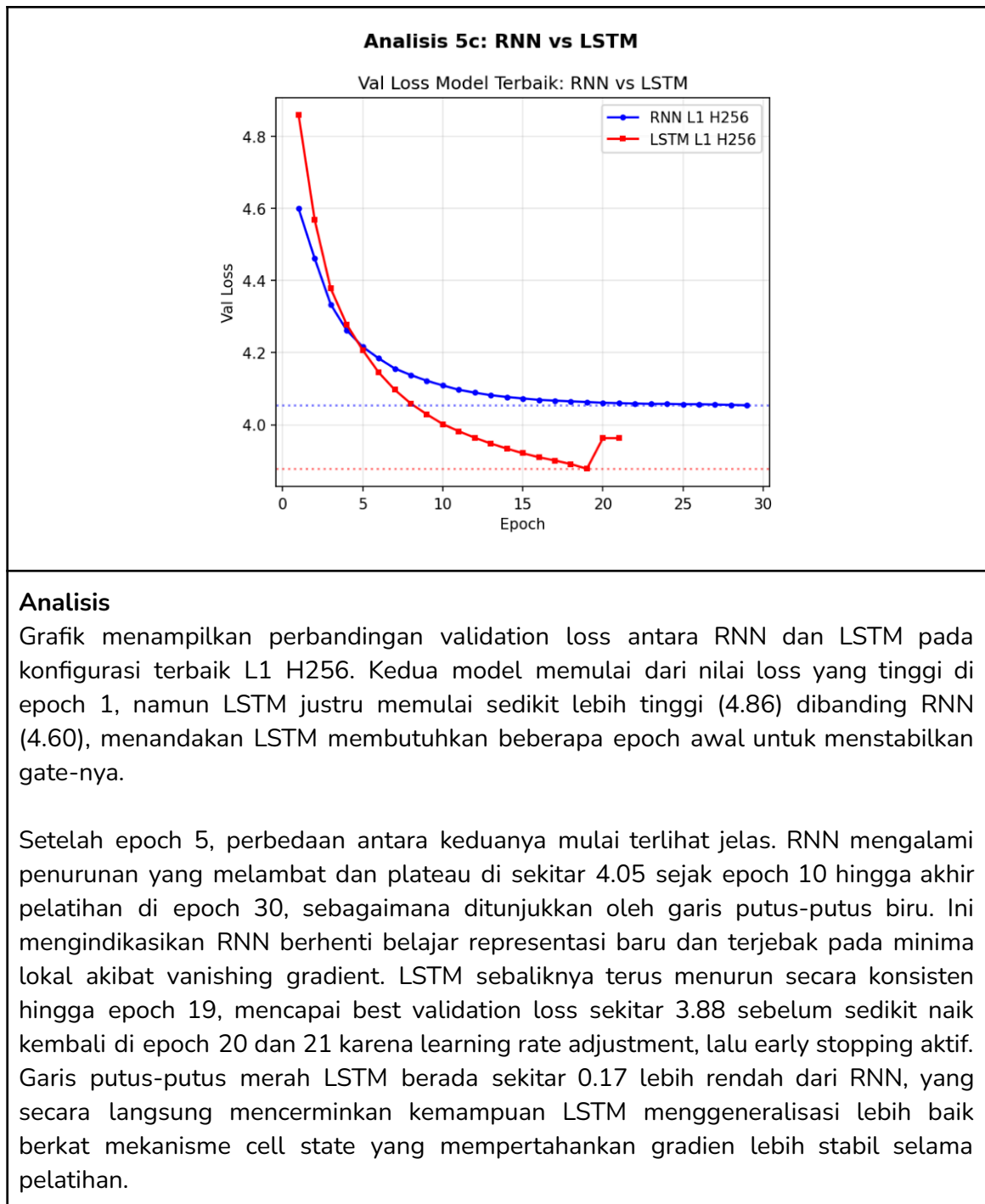


Analisis:

Dari segi score, LSTM secara konsisten unggul atas RNN pada kedua metrik. LSTM mencapai BLEU-4 0.0526 berbanding RNN 0.0291. Pada METEOR, LSTM menghasilkan 0.2719 berbanding RNN 0.2575. Perbedaan BLEU-4 yang jauh lebih besar dibanding METEOR menunjukkan bahwa LSTM lebih unggul dalam menghasilkan urutan n-gram yang tepat, sementara pada level kata tunggal dan sinonim kedua model tidak terlalu jauh berbeda.

Dari segi waktu inferensi, RNN lebih cepat dibanding LSTM pada kedua implementasi. RNN Scratch hanya membutuhkan 3.5ms per gambar, sedangkan LSTM Scratch membutuhkan 5.9ms, sekitar 1.7 kali lebih lambat. Pada implementasi Keras, RNN membutuhkan 991.9ms dan LSTM 1384.3ms per gambar, dengan pola rasio yang serupa. Perbedaan kecepatan ini wajar karena LSTM memiliki empat gate yang harus dikomputasi di setiap timestep dibanding satu transformasi linear pada RNN. Meski demikian, trade-off kecepatan sebesar 1.7 kali tersebut sangat sepadan mengingat peningkatan BLEU-4 yang mencapai 80.8%.

ii. Training dan Validation Loss



iii. Qualitative Analysis

```
[{'image': '3721799573_2f470950e0.jpg',
  'rnn_bleu4': 9.55442792204367e-08,
  'lstm_bleu4': 9.365137582048793e-08,
  'rnn_caption': 'a woman in a black shirt and a is hat a a',
  'lstm_caption': 'a man in a blue shirt and a is is in a of',
```

'ground_truth': ['There are two ladies embracing and posing shown here .',
 'Two girlfriends smiling .']],
 {'image': '3725177385_62d5e13634.jpg',
 'rnn_bleu4': 1.4388178057096277e-07,
 'lstm_bleu4': 1.2091070234069986e-07,
 'rnn_caption': 'a race car driver through a turn',
 'lstm_caption': 'a man is a a on on motorcycle motorcycle',
 'ground_truth': ['A bunch of vintage cars are parked in a lot .',
 'An antique car that is yellow is shown lined up with other cars .']],
 {'image': '3729405438_6e79077ab2.jpg',
 'rnn_bleu4': 1.6548754598234378e-07,
 'lstm_bleu4': 1.2014057070673781e-05,
 'rnn_caption': 'a man is a a on on bike',
 'lstm_caption': 'a man in a blue shirt and a pants shirt a a on on skateboard
 skateboard',
 'ground_truth': ['A bicycler balances his bike on a rail .',
 'A biker is riding down the stairs on his bike at night .']],
 {'image': '3721881082_afe9fc734e.jpg',
 'rnn_bleu4': 1.1645788017799575e-07,
 'lstm_bleu4': 1.7167035531989437e-05,
 'rnn_caption': 'a man of a heritage wearing on black',
 'lstm_caption': 'a man and a woman are on on sidewalk street a a',
 'ground_truth': ['A woman and a girl sitting on a front lawn',
 '"A woman and girl wearing traditional clothes , sits on the front lawn , reading
 from a book ."]'],
 {'image': '3725814794_30db172f67.jpg',
 'rnn_bleu4': 1.351200154807034e-05,
 'lstm_bleu4': 1.5352597838656372e-05,
 'rnn_caption': 'a man player a a and ball is a a',
 'lstm_caption': 'a basketball player in a is uniform a the',
 'ground_truth': ['A man performing martial arts kicks his opponent .',
 '"During a martial arts match , a man attempts to kick the other in the face ."]'],
 {'image': '3728256505_7f8db8270d.jpg',
 'rnn_bleu4': 1.8092176081223304e-05,
 'lstm_bleu4': 1.4554368293886241e-05,
 'rnn_caption': 'a man is a a on on bike',
 'lstm_caption': 'a man in a blue shirt is a a on on skateboard skateboard',
 'ground_truth': ['A man jumping onto a platform .',
 'A man on the edge of a wall about to fall off .']],
 {'image': '3726700898_c50494b8bd.jpg',
 'rnn_bleu4': 1.8046314683044433e-05,
 'lstm_bleu4': 1.6548754598234387e-05,

```

'rnn_caption': 'a man in a black shirt and a is hat a a',
'lstm_caption': 'a man in a blue shirt and a is is on a back in a air',
'ground_truth': ['A man balances a knife on his arm as another man watches .',
'A man has a knife on his arm and another man is watching .']],
{'image': '3721812313_6000566803.jpg',
'rnn_bleu4': 0.0016653831334339092,
'lstm_bleu4': 1.5461103513878924e-07,
'rnn_caption': 'a man of a woman wearing a white hat and a is hat a a',
'lstm_caption': 'a man of a and and woman woman a a pose in a of',
'ground_truth': ['A closeup shot of three guys and a girl posing for the camera .',
'A woman and three men are posing for a photo']],
{'image': '3724718895_bd03f4a4dc.jpg',
'rnn_bleu4': 1.9304869754804474e-05,
'lstm_bleu4': 0.38999752528799086,
'rnn_caption': 'a dog and a dog dog in a snow',
'lstm_caption': 'a black and white dog is running through a grass field a a',
'ground_truth': ['a dog drinks the sprinkler water .',
'A small brown and white dog is playing with the sprinkler in the grass .']],
{'image': '3726025663_e7d35d23f6.jpg',
'rnn_bleu4': 0.30213753973567675,
'lstm_bleu4': 0.31239399369202553,
'rnn_caption': 'a group of people are a a on on motorcycle',
'lstm_caption': 'a group of people are a bicycles on a dirt',
'ground_truth': ['A group of people ride in a race',
'A large group of bicycles riding their bikes .']}]

```

Analisis

Dari 10 contoh kualitatif, LSTM secara umum menghasilkan caption yang lebih baik, meskipun dengan beberapa pengecualian. Pada kasus dengan skor sangat rendah (contoh 1 dan 2), kedua model gagal menghasilkan caption yang relevan. RNN cenderung menghasilkan token berulang dan tidak bermakna seperti "a is hat a a", sedangkan LSTM mengalami pola serupa dengan "is is" dan "on on". Namun pada contoh 3 dan 4, LSTM mulai menunjukkan keunggulan dengan skor lebih tinggi dari RNN, meskipun caption-nya masih mengandung repetisi. LSTM setidaknya mampu menyebut atribut yang lebih relevan seperti warna baju dan konteks lokasi.

Pada rentang skor menengah (contoh 5 hingga 7), kedua model masih kesulitan menangkap konteks yang tepat. Menariknya pada contoh 8, RNN justru unggul drastis dengan BLEU-4 0.00167 berbanding LSTM yang hanya 1.5e-7, karena RNN secara kebetulan menghasilkan beberapa kata yang cocok dengan ground truth sementara LSTM mengulang kata "woman woman" dan "and and" secara tidak koheren.

Kasus paling mencolok adalah contoh 9, di mana LSTM mencapai BLEU-4 0.390 berbanding RNN yang hanya 1.9e-5. Caption LSTM "a black and white dog is running through a grass field" jauh lebih akurat dan koheren dibanding RNN yang menghasilkan "a dog and a dog dog in a snow". Contoh 10 juga menunjukkan LSTM sedikit lebih baik dengan menyebut "bicycles" yang lebih spesifik dibanding "motorcycle" milik RNN.

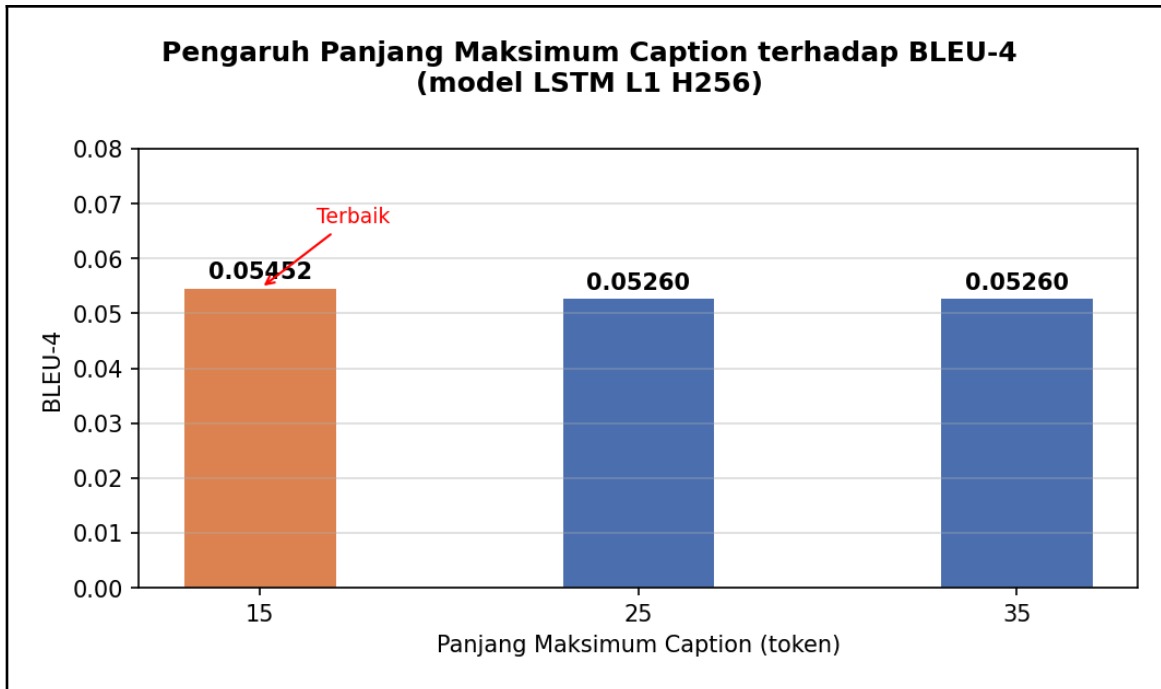
Secara keseluruhan, pola yang teridentifikasi adalah RNN lebih sering menghasilkan caption sangat pendek atau repetitif akibat ketidakmampuan mempertahankan konteks antar token, sedangkan LSTM lebih sering menghasilkan struktur kalimat yang lebih lengkap dan atribut visual yang lebih relevan, meskipun repetisi masih sesekali muncul.

iv. Kesimpulan

LSTM secara konsisten unggul atas RNN. Grafik validation loss menunjukkan LSTM terus menurun hingga epoch 19 dengan best val loss 3.88, jauh lebih rendah dibanding RNN yang plateau di 4.05 sejak epoch 10. Analisis kualitatif mengonfirmasi bahwa LSTM menghasilkan caption yang lebih koheren dan kaya atribut visual, sementara RNN lebih sering menghasilkan token repetitif akibat ketidakmampuan mempertahankan konteks antar token. Keunggulan LSTM berakar pada mekanisme cell state dan gating yang mengatasi vanishing gradient, memungkinkan retensi konteks visual lebih lama selama proses decoding. Trade-off kecepatan sebesar 1.7 kali sangat sepadan dengan peningkatan performa yang signifikan.

d. Pengaruh panjang maksimum caption

i. BLEU-4 Score



Analisis:

Grafik menunjukkan bahwa panjang maksimum caption berpengaruh terhadap BLEU-4, namun hanya pada rentang tertentu. Max length 15 token menghasilkan BLEU-4 tertinggi sebesar 0.05452, sedangkan max length 25 dan 35 keduanya menghasilkan nilai yang sama persis yaitu 0.05260, turun sekitar 3.5% dari konfigurasi terbaik.

Kesamaan nilai antara max length 25 dan 35 mengindikasikan bahwa model LSTM L1 H256 secara alami menghasilkan token stop sebelum mencapai batas 25 token pada sebagian besar gambar, sehingga menambah batas lebih jauh ke 35 tidak mengubah caption yang dihasilkan sama sekali. Sementara itu, keunggulan max length 15 dapat dijelaskan oleh karakteristik dataset Flickr8k yang ground truth caption-nya rata-rata pendek sekitar 10 hingga 12 token. Dengan batas yang lebih ketat, model terdorong untuk langsung menghasilkan informasi visual yang paling esensial tanpa ruang untuk token pengisi yang tidak relevan, sehingga precision n-gram yang diukur BLEU-4 menjadi lebih tinggi.

ii. Kesimpulan

Panjang maksimum caption 15 token menghasilkan BLEU-4 terbaik sebesar 0.05452 pada model LSTM L1 H256. Panjang 15 token optimal karena sesuai dengan distribusi ground truth Flickr8k yang rata-rata pendek, sehingga model terdorong menghasilkan informasi visual yang esensial tanpa token pengisi yang menurunkan precision BLEU-4.

BAB III

KESIMPULAN DAN SARAN

3.1 Kesimpulan

Berdasarkan implementasi dan pengujian yang telah dilakukan, diperoleh beberapa kesimpulan sebagai berikut:

- Pipeline CNN untuk klasifikasi citra dan pipeline RNN/LSTM untuk image captioning berhasil diimplementasikan menggunakan Keras dan implementasi from scratch berbasis NumPy.
- Pada bagian CNN, model dengan shared parameter memberikan hasil lebih baik dibandingkan non-shared parameter. Model shared terbaik memperoleh macro F1-score sekitar 0.789, sedangkan model non-shared memperoleh macro F1-score sekitar 0.644 dengan jumlah parameter yang jauh lebih besar.
- Variasi hyperparameter CNN menunjukkan bahwa konfigurasi dengan dua layer konvolusi, jumlah filter lebih besar, kernel 3x3, dan max pooling memberikan hasil terbaik untuk klasifikasi citra.
- Implementasi forward propagation CNN from scratch menghasilkan prediksi yang konsisten dengan Keras, sehingga proses load bobot dan alur komputasi layer dapat dikatakan sudah sesuai.
- Pada bagian image captioning, LSTM memberikan performa lebih baik dibandingkan Simple RNN berdasarkan BLEU-4, METEOR, dan validation loss. Hal ini menunjukkan bahwa mekanisme gate dan cell state pada LSTM lebih efektif dalam mempertahankan konteks antar token.
- Implementasi RNN/LSTM from scratch menghasilkan score yang sama dengan Keras karena menggunakan bobot yang sama. Dengan demikian, implementasi forward propagation from scratch sudah konsisten dengan model Keras.
- Secara keseluruhan, Keras memudahkan proses training dan eksperimen, sedangkan implementasi from scratch membantu memahami detail perhitungan di dalam CNN, RNN, dan LSTM.

3.2 Saran

Beberapa saran yang dapat dilakukan untuk pengembangan selanjutnya adalah sebagai berikut:

- Performa CNN dapat ditingkatkan dengan augmentasi data, regularisasi, batch normalization, dropout, atau transfer learning menggunakan model pretrained seperti ResNet dan EfficientNet.
- Evaluasi forward propagation CNN from scratch dapat dilakukan pada jumlah sampel test yang lebih besar agar validasi terhadap model Keras menjadi lebih kuat.
- Kualitas image captioning dapat ditingkatkan dengan menggunakan encoder CNN yang lebih kuat serta menambahkan mekanisme attention agar decoder dapat fokus pada bagian gambar yang relevan.
- Preprocessing caption dapat diperbaiki melalui eksplorasi vocabulary size, tokenisasi yang lebih baik, dan penanganan rare words.
- Tuning hyperparameter seperti learning rate, batch size, jumlah epoch, dan early stopping dapat dilakukan lebih sistematis agar performa model lebih optimal.
- Implementasi from scratch dapat dikembangkan lebih lanjut hingga mendukung proses training penuh, sehingga model tidak hanya digunakan untuk inferensi tetapi juga dapat dilatih tanpa bergantung pada Keras.

Pembagian Tugas Tiap Anggota Kelompok

Nama Lengkap	NIM	Tugas
Indah Novita Tangdililing	13523047	Membuat caption preprocessing, feature extraction Flickr8k, membuat RNN from scratch, membuat LSTM from scratch, membuat training decoder RNN dan LSTM, melakukan evaluasi captioning dengan BLEU-4 dan METEOR, membuat qualitative analysis hasil caption.
Muhammad Fithra Rizki	13523049	Menyusun infrastruktur project, melakukan evaluasi framework, membuat plotting dan visualisasi hasil eksperimen, mengintegrasikan pipeline CNN dan RNN/LSTM, membuat batch inference pipeline, mendokumentasikan README dan laporan, implementasi bonus visualisasi fitur CNN dan Grad-CAM, implementasi bonus init-inject, implementasi bonus beam search decoder, implementasi bonus batch inference, implementasi bonus backward propagation from scratch.
Muhammad Timur Kanigara	13523055	Membuat Utility image processing, membuat CNN from scratch, melakukan CNN Keras training, melakukan eksperimen hyperparameter CNN, membuat evaluasi CNN, melakukan perbandingan shared vs non-shared parameter.

Referensi

Tim Pengajar IF3270. Recurrent Neural Network (ANN) Part 1 dan 2. Diakses dari: edunex

Tim Pengajar IF3270. Convolutional Neural Network. Diakses dari: edunex

Selvaraju, R. R., Cogswell, M., Das, A., Vedantam, R., Parikh, D., & Batra, D. (2017). Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization. Diakses dari: <https://arxiv.org/abs/1610.02391>

Tanti, M., Gatt, A., & Camilleri, K. P. (2017). Where to put the Image in an Image Caption Generator. Diakses dari: <https://arxiv.org/abs/1703.09137>

Zhang, A., Lipton, Z. C., Li, M., & Smola, A. J. (2023). Beam Search. Dalam Dive into Deep Learning. Diakses dari: https://d2l.ai/chapter_recurrent-modern/beam-search.html

Zhang, A., Lipton, Z. C., Li, M., & Smola, A. J. (2023). Recurrent Neural Network Implementation from Scratch. Dalam Dive into Deep Learning. Diakses dari: https://d2l.ai/chapter_recurrent-neural-networks/rnn-scratch.html

Keras Team. (2026). Image Captioning. Diakses dari: https://keras.io/examples/vision/image_captioning/

FORMAT DEKLARASI PENGGUNAAN AI

SURAT PERNYATAAN PENGGUNAAN AI

Dengan ini, kami:

Nama/NIM : Indah Novita Tangdililing (13523047)
 Muhammad Fithra Rizki (13523049)
 Muhammad Timur Kanigara (13523055)

Fakultas/Sekolah : STEI

Kode Mata Kuliah : IF3270

Nama Mata Kuliah : Pembelajaran Mesin

Judul Tugas/Proposal : Tugas Besar 2 Pembelajaran Mesin

Menyatakan bahwa kami menggunakan AI dalam pengerjaan maupun penyusunan luaran tugas pada mata kuliah yang tertulis di atas.

Jika Ya, maka alat AI/kecerdasan buatan yang digunakan adalah:

Lingkup Pekerjaan	Digunakan?	Tingkat Penggunaan
Pemeriksaan Ejaan: menggunakan tools seperti Grammarly, Paperpal, Deepl, Quillbot, Grammarbot, LanguageTool, ProWritingAid, ChatGPT, Google Gemini, atau sejenisnya.	Tidak	Tingkat 1
Pembuatan Teks: menggunakan tools seperti ChatGPT, Gemini, Paperpal, Copilot, GrammarlyGO, WordAI, WriteSonic, Jasper, Jenni AI, atau sejenisnya.	Tidak	Tingkat 1
Bantuan Diskusi dalam Penyusunan Konten: menggunakan tools seperti ChatGPT, Gemini, Copilot, Perplexity, Jenni AI, Zoom-Companion, atau sejenisnya.	Ya	Tingkat 2
Pembuatan Gambar, Video, dan/atau Grafik: menggunakan tools seperti Craiyon, DALL-E, Midjourney, Stable Diffusion, Microsoft Designer, Gemini, Canva AI, atau sejenisnya.	Tidak	Tingkat 1

Lainnya, Jelaskan:		
---------------------------	--	--

Kami juga menyatakan bahwa setiap penggunaan AI/kecerdasan buatan yang dilakukan pada mata kuliah tersebut telah dinyatakan di dalam surat ini.

Bandung, 16 Mei 2026



Indah Novita Tangdililing

13523047



Muhammad Fithra Rizki

13523049



Muhammad Timur Kanigara

13523055